Gaming Dashboard Hardware Alert System

Jonathan A. Marrero Bermudez

May 4, 2026

1 Introduction

This project is a C++ hardware dashboard that uses a Raspberry Pi to monitor and display performance of a Windows PC with an NVDIA GPU. The system consists of a Windows PC sender program that collects telemetry data such as FPS, CPU usage, GPU usage, and RAM usage, and sends that data to a Raspberry Pi receiver program over UDP. The Raspberry Pi processes the incoming telemetry data and displays it on a 2.2 inch ILI9341 SPI TFT display.

The motivation behind this project is to demonstrate C++ programming fundamentals through a practical hardware system while gaining more experience programming in C++ for embedded hardware. Rather than building a purely software-based application, this project connects C++ code to real hardware components. This makes the project useful for practicing object-oriented programming, dynamic memory management, resource ownership, networking, and hardware communication in a more realistic environment.

The project also provides experience with embedded-style development, where software must interact directly with hardware resources such as GPIO pins, SPI devices, and network sockets. By building a PC telemetry dashboard, the project connects C++ programming concepts to a practical system that receives real performance data, processes it, and displays it on external hardware. This helps strengthen my understanding of how C++ can be used not only for general software development, but also for systems that involve hardware control and resource management.

2 Software Design and Class Structure

The software design for this project was developed in stages. The first goal was not to immediately build the final dashboard, but to first create a reliable development environment and prove that the Raspberry Pi could receive data from a Windows PC. After that communication layer was working, the project expanded into a hardware display system using a TFT screen.

2.1 Raspberry Pi Development Setup

The first step in the methodology was setting up the Raspberry Pi as the embedded hardware platform. Raspberry Pi OS was installed onto a microSD card using the Raspberry Pi Imager. During this setup, SSH was enabled so that the Raspberry Pi could be accessed without requiring a dedicated monitor, keyboard, or mouse. After the Raspberry Pi booted successfully, it was connected to the same local network as the Windows PC.

Development was done remotely using SSH through Visual Studio Code. This made it possible to write and edit C++ files on the Raspberry Pi from the main computer while still compiling and running the code directly on the Raspberry Pi. This was important because the Raspberry Pi uses a different processor architecture than the Windows PC, so the Raspberry Pi program needed to be compiled on the Pi itself.

After connecting through SSH, the required development tools and hardware libraries were installed. These included the C++ compiler, build tools, SPI support, and the GPIO library used to control hardware pins. SPI was also enabled on the Raspberry Pi because the TFT display communicates using the SPI protocol.

The first software iteration was a simple C++ test program that verified the compiler and runtime environment were working. After that, the next version created a UDP receiver on the Raspberry Pi. At

this stage, the Raspberry Pi did not need to display anything on the TFT screen yet. It only needed to prove that simulated telemetry data could be received from the PC and printed to the terminal. This step helped separate the networking problem from the hardware display problem.

2.2 Initial Telemetry Receiver Design

The Raspberry Pi receiver was designed around a simple telemetry packet format:

```
FPS,CPU,GPU,RAM
```

For example, a packet such as `144,42,78,61` represents 144 FPS, 42% CPU usage, 78% GPU usage, and 61% RAM usage.

The first major class created for the Raspberry Pi side was a telemetry data class. This class stores the values received from the PC.

- `TelemetryData` stores FPS, CPU usage, GPU usage, and RAM usage.
- `TelemetryReceiver` acts as an abstract base class for receiving telemetry.
- `UdpTelemetryReceiver` derives from `TelemetryReceiver` and implements UDP socket communication.

The purpose of this structure was to separate the idea of "receiving telemetry" from the specific method used to receive it. This demonstrates abstraction because `TelemetryReceiver` defines what a receiver should do without requiring the rest of the program to know the details of UDP sockets. The derived `UdpTelemetryReceiver` class provides the actual implementation.

This structure also demonstrates inheritance and polymorphism. The main program can store a receiver using a base class pointer such as `std::unique_ptr<TelemetryReceiver>`, while the actual object is a `UdpTelemetryReceiver`. This allows the program to call the receiver's `receive()` function through the base class interface.

Dynamic memory management is demonstrated through the use of `std::unique_ptr`. Instead of manually using `new` and `delete`, the project uses smart pointers to manage dynamically allocated objects safely. This avoids memory leaks and clearly shows ownership of dynamically created objects.

Resource ownership is also demonstrated in the `UdpTelemetryReceiver` class. The class owns the UDP socket file descriptor. Its constructor creates and binds the socket, while its destructor closes the socket. This follows the RAII principle, where a resource is acquired during object construction and released during object destruction.

2.3 Windows PC Telemetry Sender

After the Raspberry Pi receiver was tested with simulated packets, the next step was to create the Windows PC sender program, `send_data.cpp`. The purpose of this program is to collect hardware telemetry from the PC and send it to the Raspberry Pi over the local network using UDP.

The sender program was first written using simulated random values. This made it easier to test whether the network transfer worked before adding real hardware monitoring. Once the transfer was working, real PC values were added step by step.

The Windows sender is responsible for:

- Collecting CPU usage.
- Collecting RAM usage.
- Collecting NVIDIA GPU usage.
- Formatting the values into a telemetry string.
- Sending the string to the Raspberry Pi through a UDP socket.

CPU and RAM usage were collected using Windows system functions. GPU usage was collected using NVIDIA tools because GPU monitoring is more vendor-specific than CPU or RAM monitoring. FPS was left as a simulated value and listed as a future improvement, because real FPS monitoring usually requires tools such as PresentMon, RTSS, or game-specific frame presentation tracking.

The sender program also demonstrates resource ownership. It initializes the Windows socket system, creates a UDP socket, sends telemetry packets, and then cleans up the socket resources when the program ends. This makes the sender side conceptually similar to the Raspberry Pi receiver: both programs use C++ objects and system resources to manage communication.

2.4 UDP Communication Methodology

After the Windows sender was completed, the next major step was setting up and debugging communication between the PC and the Raspberry Pi. UDP was chosen because the project sends live telemetry data. If one packet is missed, a newer packet will arrive shortly after, so the system does not require the reliability overhead of TCP.

The Raspberry Pi listens on UDP port 5000. The Windows sender sends packets to the Raspberry Pi's local IP address on that port. Communication was first tested using PowerShell to send a known test packet. This helped confirm that the Raspberry Pi receiver worked before relying on the full C++ sender program.

The use of UDP sockets supports the networking requirement of the project while also giving a practical example of resource management in C++. Sockets are operating system resources that must be opened, used, and closed properly. Encapsulating this behavior inside classes makes the code easier to manage and explain.

2.5 Adding the TFT Display

After telemetry transfer was working, a TFT display was added to make the project more useful and visually interesting. Instead of only printing telemetry values to the terminal, the Raspberry Pi could act as a small desktop companion device that shows live PC performance data on a dedicated screen.

The display used in the project is a 2.2 inch 240x320 SPI TFT display using the ILI9341 controller. The ILI9341 controller is commonly used for small 240x320 TFT displays, and its command set includes operations for setting the address window, writing pixel data, selecting pixel format, and turning the display on. The ILI9341 datasheet describes the controller as supporting a 240xRGB by 320 display resolution and software-controlled features such as sleep mode and display control. [**?**]

The display communicates with the Raspberry Pi using SPI. SPI is used to send command bytes and pixel data from the Raspberry Pi to the display. Additional GPIO pins are used for control signals such as DC and RESET. The DC pin determines whether the byte being sent is a command or display data, while the RESET pin is used to reset the display during initialization.

2.6 ILI9341 Display Class

To keep the display logic organized, a separate `ILI9341Display` class was created. The idea for this class came from studying how existing ILI9341 display libraries are structured. Many ILI9341 drivers follow a similar pattern: open the hardware interface, reset the display, send an initialization sequence, set an address window, and then write pixel data. Public ILI9341 examples and libraries, such as Adafruit's ILI9341 library documentation and other SPI TFT driver examples, helped guide the general structure of the custom class. [**?**]

The custom `ILI9341Display` class was written specifically for this project rather than directly copying a complete library. It was designed to demonstrate C++ fundamentals while still being simple enough to understand. The class handles:

- Opening the SPI device.
- Requesting GPIO lines for DC and RESET.
- Sending commands and data to the display.

- Initializing the ILI9341 controller.
- Drawing pixels, rectangles, text, and usage bars.
- Displaying the telemetry dashboard.

The class structure is important because it clearly demonstrates encapsulation. The main program does not need to know the details of SPI transfers, GPIO line requests, command bytes, or address windows. Instead, the main program can simply call a function such as `showDashboard(fps, cpu, gpu, ram)`. The lower-level display behavior is hidden inside the class.

Resource ownership is also demonstrated strongly in this class. The `ILI9341Display` object owns the SPI file descriptor and the GPIO line request. Its constructor opens the SPI device, configures SPI settings, requests the required GPIO lines, and initializes the screen. Its destructor releases the GPIO request and closes the SPI file descriptor. This is another example of RAII because hardware resources are tied directly to the lifetime of the C++ object.

The display class also connects to the project requirement for constructors and destructors. The constructor performs hardware setup, and the destructor performs hardware cleanup. This gives a practical reason for using these C++ features rather than including them only for demonstration.

2.7 Overall Class Structure

The final class structure was intentionally kept simple so that the project could focus on core C++ fundamentals rather than unnecessary complexity. The main classes are `TelemetryData`, `TelemetryReceiver`, `UdpTelemetryReceiver`, and `ILI9341Display`. Each class has a specific role in the system, which makes the program easier to understand, test, and expand.

The thought process behind this structure was to separate responsibilities. `TelemetryData` acts as a simple data container for values such as FPS, CPU usage, GPU usage, and RAM usage. `TelemetryReceiver` provides a general interface for receiving telemetry data, while `UdpTelemetryReceiver` implements that interface using UDP network communication. `ILI9341Display` is responsible for controlling the TFT screen and drawing the dashboard output. The `main` function connects these parts together by creating the receiver and display objects, receiving telemetry packets, and passing the parsed data to the screen. This structure keeps networking, data storage, and hardware display logic separate while still allowing them to work together as one system.

3 C++ Concepts Demonstrated

This design demonstrates several important C++ concepts. Encapsulation is used by separating the program into classes that each hide their internal details and expose only the functions needed by the rest of the program. Abstraction is shown through the `TelemetryReceiver` class, which defines a general interface for receiving data without requiring the main program to depend directly on UDP socket details. Inheritance is demonstrated because `UdpTelemetryReceiver` inherits from `TelemetryReceiver`, and polymorphism is used when the receiver is accessed through a base class pointer, allowing the program to call `receive()` without depending on the exact derived type. Dynamic memory management is demonstrated with `std::unique_ptr`, which safely manages dynamically created receiver objects without requiring manual deletion. Constructors and destructors are also important in this design because constructors initialize resources such as sockets, SPI, GPIO, and the display, while destructors release those resources when the objects are destroyed. This connects directly to resource ownership: the receiver class owns the UDP socket, and the display class owns the SPI file descriptor and GPIO request. Overall, the class structure was designed to be simple and focused on demonstrating C++ fundamentals while still creating a functional hardware dashboard system.

4 Results

The final result of the project was a working hardware-based PC performance dashboard. The Windows PC sender successfully collected and transmitted telemetry data over UDP, and the Raspberry Pi receiver

successfully received, parsed, and displayed the data on the connected ILI9341 TFT screen. The dashboard was able to show values for FPS, CPU usage, GPU usage, RAM usage, and a basic system status indicator.

Overall, the project successfully demonstrated UDP communication between a Windows PC and a Raspberry Pi, hardware display control through SPI, and the use of C++ concepts such as classes, resource ownership, constructors, destructors, and dynamic memory management in a practical hardware system.

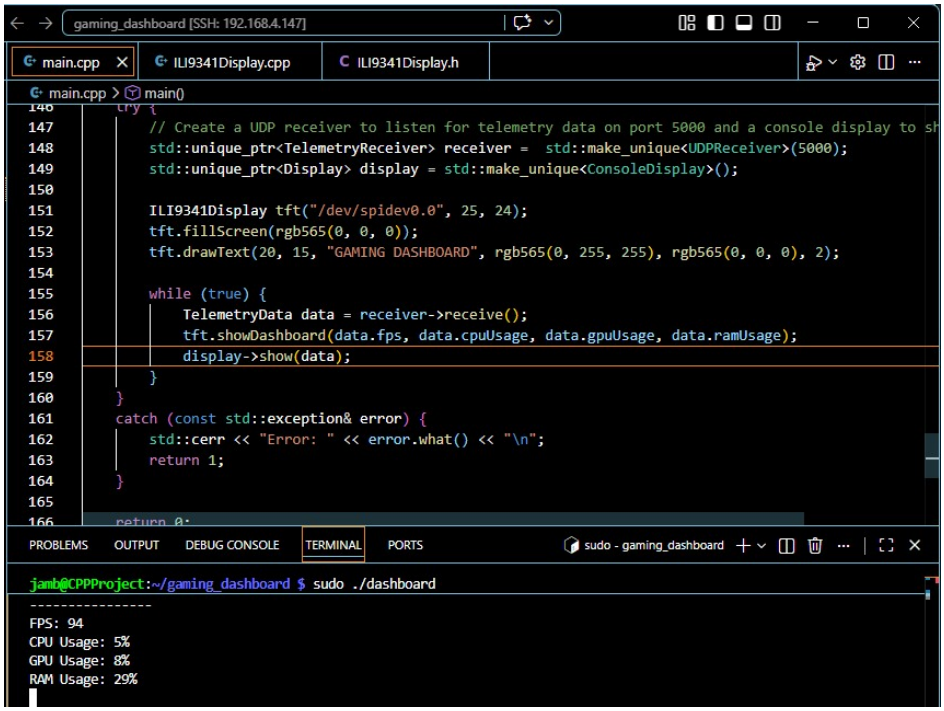

Figure 1: Raspberry Pi terminal showing received telemetry data from the Windows PC.

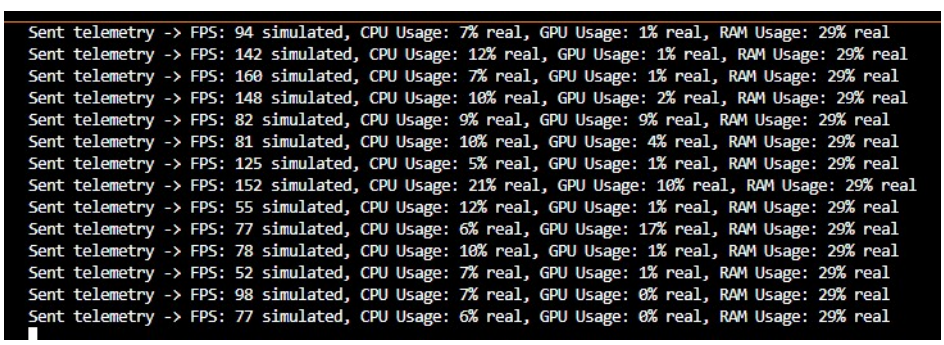

Figure 2: Windows C++ sender program transmitting telemetry packets to the Raspberry Pi.

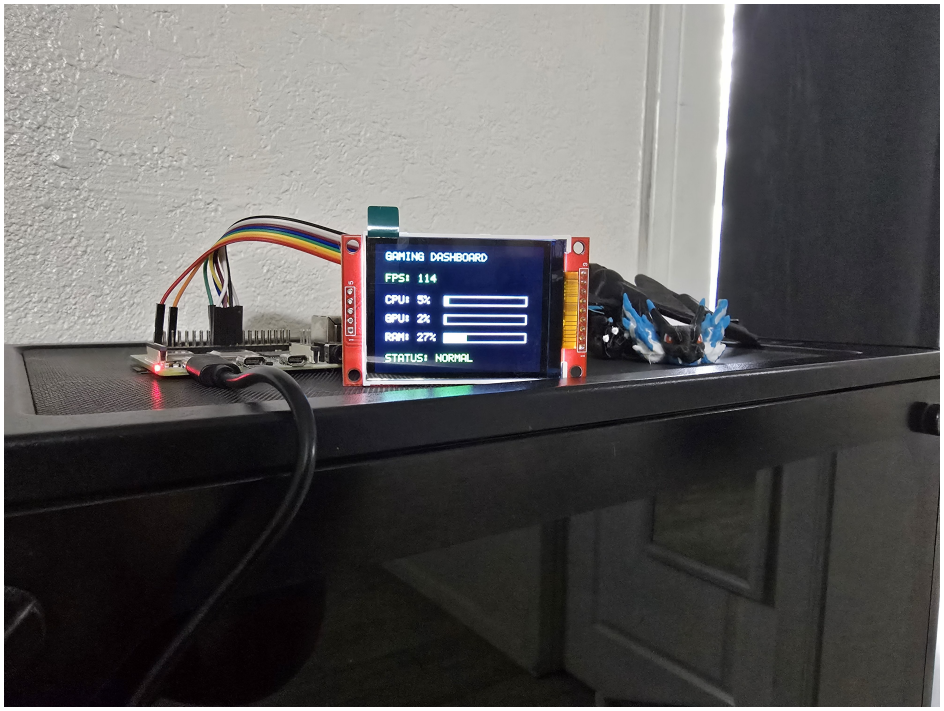

Figure 3: ILI9341 TFT display showing the real-time gaming dashboard output.

5 Conclusion

This project successfully demonstrates a C++ hardware dashboard system that receives PC telemetry data and displays it on a Raspberry Pi-connected TFT screen. The project combines networking, hardware communication, object-oriented programming, and resource ownership. It also provides a foundation for future improvements such as real FPS monitoring and expanded hardware alerts.